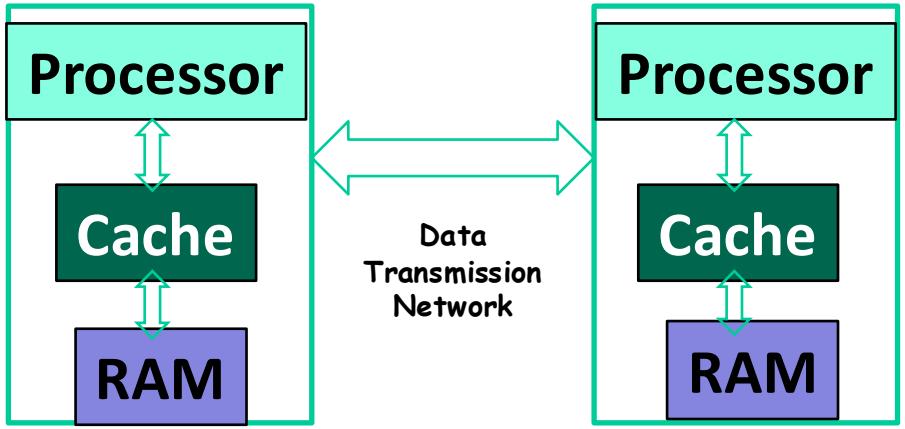
Introduction to Parallel Programming

Lecture: MPI Basic

谭光明
高性能计算机研究中心

Introduction

- The processors in the computer systems with distributed memory operate independently.
- It is necessary to have a possibility:
  - to distribute the computational load,
  - to organize the information communication (data transmission) among the processors.

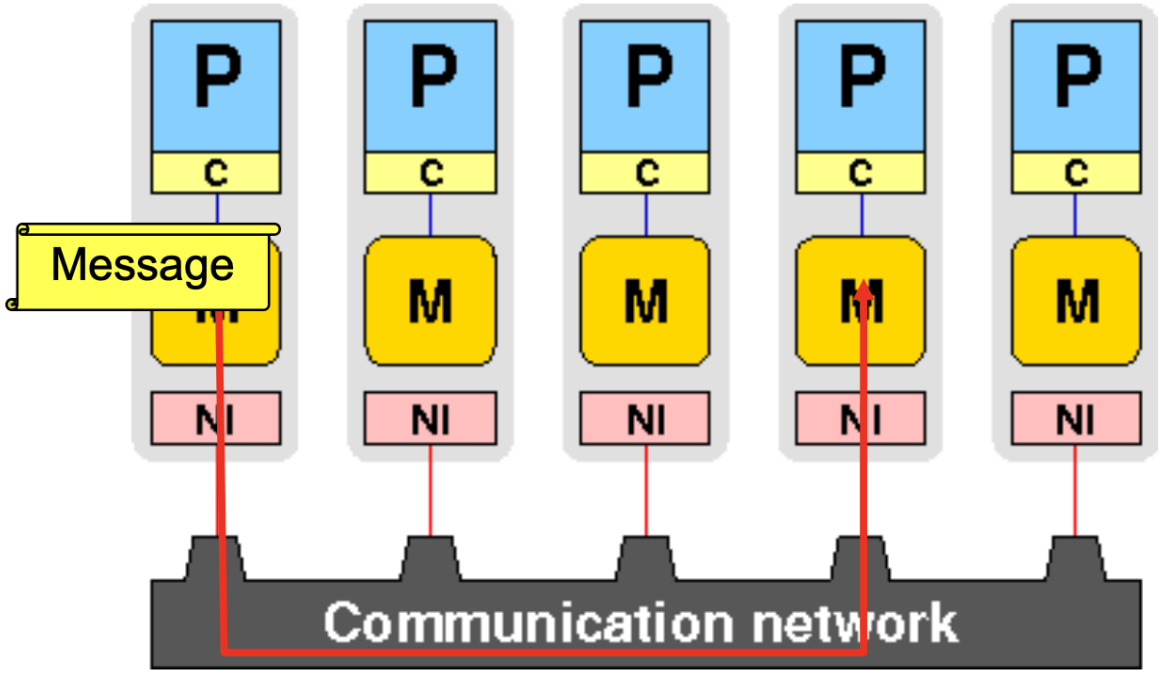


The solution of the above mentioned problems is provided by the message passing paradigm

The message passing paradigm

Distributed-memory architecture

- Process(or) can only access its dedicated address space
- No global shared address space
- Data exchange and communication between processes is done by explicitly passing messages through a communication network



Message passing library

- **Should be flexible, efficient and portable**
- **Hide communication hardware and software layers from application developer**

The message passing paradigm

- Widely accepted standard in HPC / numerical simulation
  - Message Passing Interface (MPI)
- Process-based approach: All variables are local!
- Same program on each processor/machine (SPMD)
- The program is written in a sequential language (Fortran/C[++]), but not restricted only to these two programming languages
- Data exchange between processes: Send/receive messages via MPI library calls
  - No automatic workload distribution

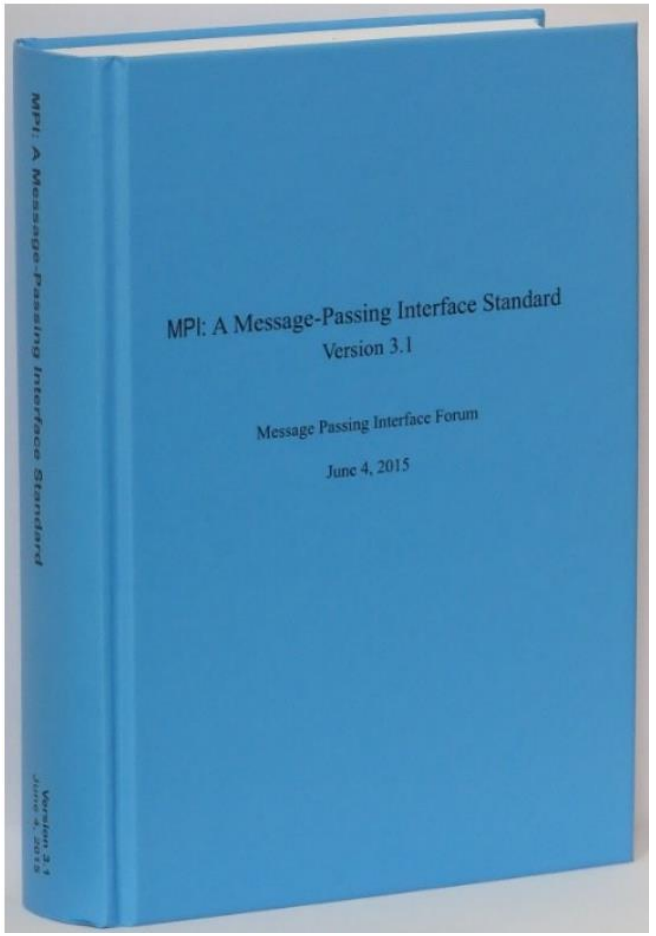
All parallelism is explicit: the programmer is responsible for correctly identifying parallelism and implementing parallel algorithms using MPI constructs

What is MPI

- MPI: Message Passing Interface
  - The MPI Forum organized in 1992 with broad participation by:
    - Vendors: IBM, Intel, TMC, SGI, Convex, Meiko
    - Portability library writers: PVM, p4
    - Users: application scientists and library writers
    - MPI-1 finished in 18 months
- Incorporates the best ideas in a "standard" way
  - Each function takes fixed arguments
  - Each function has fixed semantics
    - Standardizes what the MPI implementation provides and what the application can and cannot expect
    - Each system can implement it differently as long as the semantics match
- MPI is not…
  - a language or compiler specification
  - a specific implementation or product

The MPI standard

- MPI forum – defines MPI standard / library subroutine interfaces
- Latest standard in use: MPI 3.1 (2015), 868 pages
  - MPI-4.1 was approved by the MPI Forum on 02.11.2023
- Members (approx. 60) of MPI standard forum
  - Application developers
  - Research institutes & computing centers
  - Manufacturers of supercomputers & software designers
- Successful free implementations (MPICH, mvapich, OpenMPI) and vendor libraries (Intel, Cray, HP,…)
- Documents: http://www.mpi-forum.org/



MPI-1

- MPI-1 supports the classical message-passing programming model: basic point-to-point communication, collectives, datatypes, etc
- MPI-1 was defined (1994) by a broadly based group of parallel computer vendors, computer scientists, and applications developers.
  - 2-year intensive process
- Implementations appeared quickly and now MPI is taken for granted as vendor-supported software on any parallel machine.
- Free, portable implementations exist for clusters and other environments (MPICH, Open MPI)

Following MPI Standards

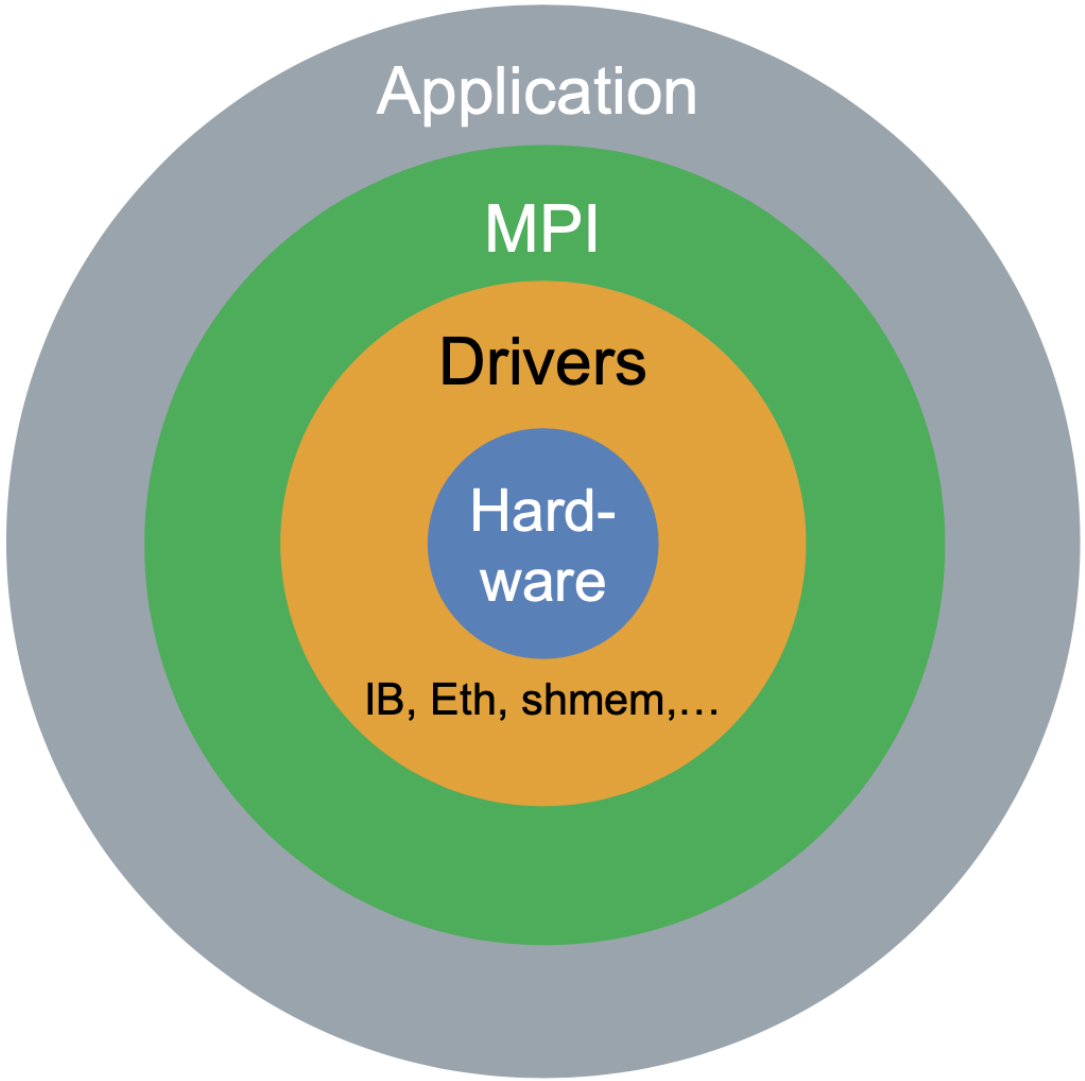
- MPI-2 was released in 1997
  - Several additional features including MPI + threads, MPI-I/O, remote memory access functionality and many others
- MPI-2.1 (2008) and MPI-2.2 (2009) were released with some corrections to the standard and small features
- MPI-3 (2012) added several new features to MPI
- MPI-3.1 (2015) introduced minor corrections and features
- MPI-4 (June 2021) was the next major update of the standard
- MPI-4.1 (Nov. 2023) was recently released with minor updates

Web Pointers

- MPI standard : https://www.mpi-forum.org/docs/
- MPI Forum : https://www.mpi-forum.org/
- MPI implementations:
  - MPICH : https://www.mpich.org
  - MVAPICH : https://mvapich.cse.ohio-state.edu/
  - Intel MPI: https://software.intel.com/en-us/intel-mpi-library/
  - Microsoft MPI: https://docs.microsoft.com/en-us/message-passing-interface/microsoft-mpi
  - Open MPI : https://www.open-mpi.org/
  - HPE/Cray MPI, IBM MPI, …
- Several MPI tutorials can be found on the web

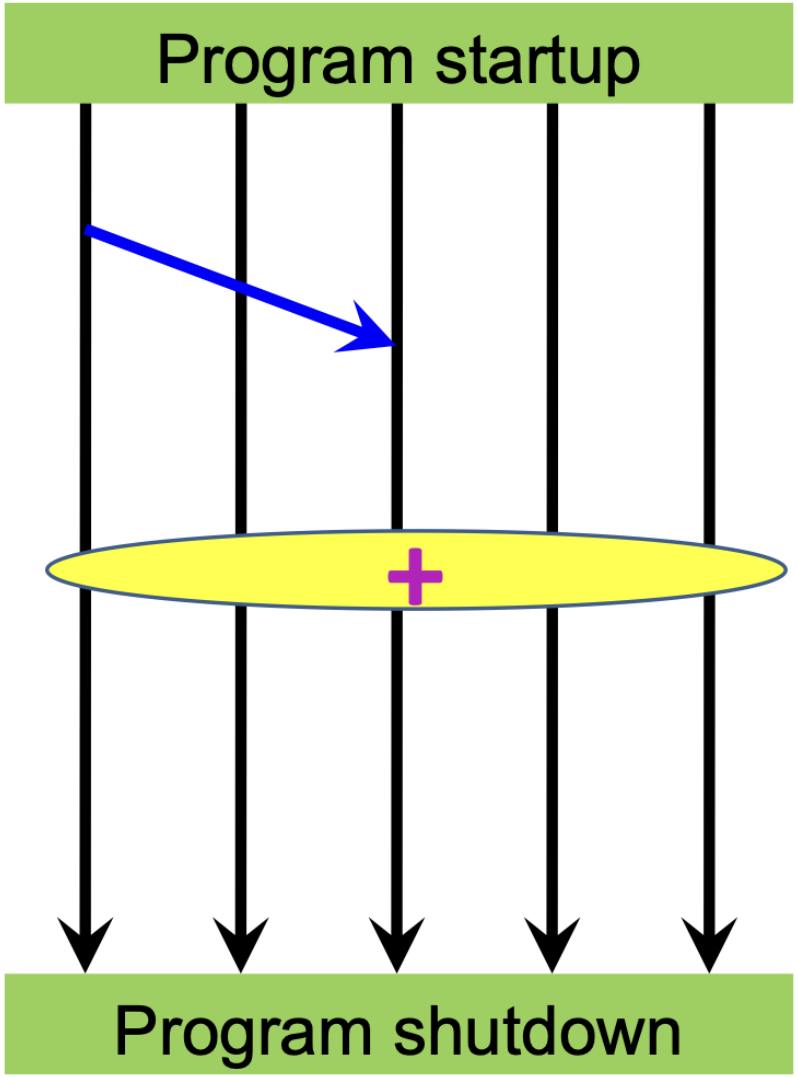
MPI goals and scope

- Portability is main goal: architecture- and hardware-independent code
- Fortran and C interfaces (C++ deprecated)
- Features for supporting parallel libraries
- Support for heterogeneous environments (e.g., clusters with compute nodes of different architectures)



Parallel execution in MPI

- Processes run throughout program execution
- MPI startup mechanism:
  - launches tasks/processes
    - think of executing multiple copies of a program
  - establishes communication context ("communicator")
- MPI Point-to-point communication:
  - between pairs of tasks/processes
- MPI Collective communication:
  - between all processes or a subgroup
  - barrier, reductions, scatter/gather
- Clean shutdown by MPI



Initialization and finalization

- Startup command of an MPI application is implementation dependent
- First call in an MPI program: initialization of parallel machine

```
int MPI_Init(int *argc, char ***argv);
```

- Last call: clean shutdown of parallel machine

```
int MPI_Finalize();
```

- Only "master" process is guaranteed to continue after finalize
- Stdout/stderr of each MPI process
  - usually redirected to console where program was started
  - many options possible, depending on implementation

Communicator and rank

- Communicator defines a set of processes (MPI_COMM_WORLD: all)
- rank: an integer identifying each process within a communicator
  - Obtain rank:

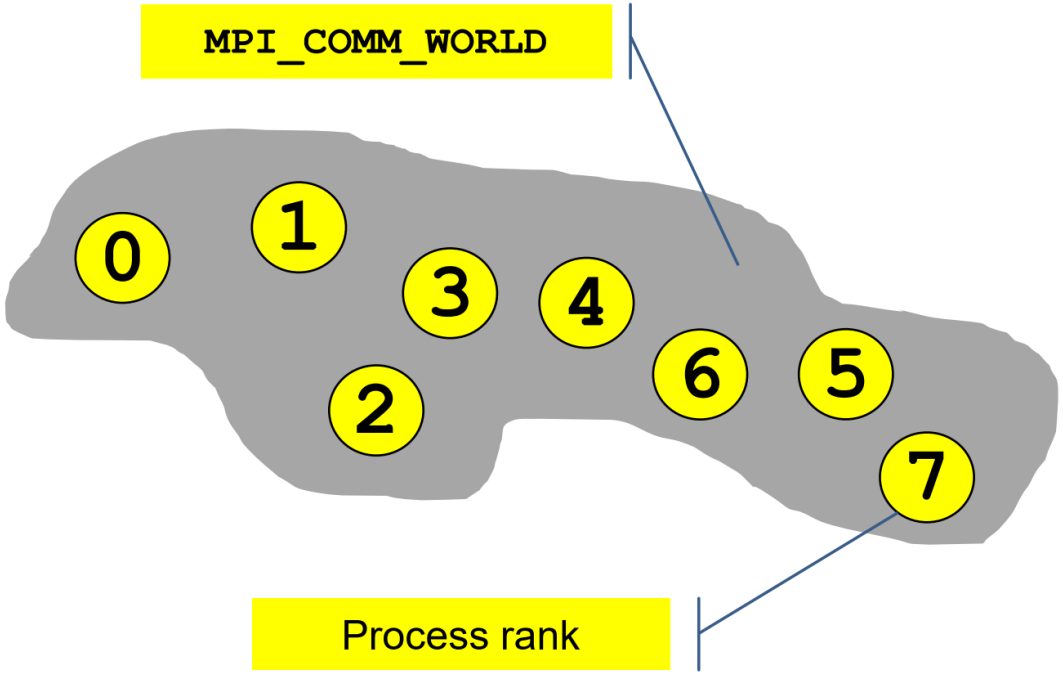
```
int rank;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

  - rank = 0,1,2,…, (number of processes in communicator - 1)
  - Not unique: one process may have distinct ranks in different communicators
- Obtain number of processes in communicator:

```
int size;
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

World communicator and rank

- Entities must be in a group/community to be able to communicate.
- Communicator is a handle
- MPI_Init():
  - MPI_COMM_WORLD
  - All processes
- MPI_COMM_WORLD
  - Fortran and C[++]



MPI "Hello World!" in C

```
/*
 * hello.c - MPI "hello, world" program
 */
#include <mpi.h>
int main(char argc, char **argv) {
        int rank, size;

        MPI_Init(&argc, &argv);

        MPI_Comm_size(MPI_COMM_WORLD, &size);
        MPI_Comm_rank(MPI_COMM_WORLD, &rank);

        printf("Hello World! I am %d of %d\n", rank, size);

        MPI_Finalize();
}
```

Never forget that these are pointers to the original variables

Communicator required for (almost) all MPI calls

Compiling and running the code

- Compiling/linking
  - Headers and libs must be found by compiler
  - Most implementations provide wrapper scripts, e.g.,
    - mpif77 / mpif90
    - mpicc / mpiCC
  - Behave like normal compilers/linkers
- Running
  - Details are implementation specific
  - Startup wrappers: mpirun, mpiexec, aprun, poe
  - Job scheduler wrappers: srun

```
$ mpiCC -o hello hello.cc
$ mpirun -np 3 ./hello
Hello World! I am 2 of 3
Hello World! I am 1 of 3
Hello World! I am 0 of 3
```

```
$ mpirun -np 1 ./hello :
-np 1 ./hello : -np 1
./hello
Hello World! I am 1 of 3
Hello World! I am 0 of 3
Hello World! I am 2 of 3
```

Basic Datatypes (C/C++)

MPI Datatype	C Datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	无对应类型
MPI_PACKED	无对应类型

Point-to-Point Communication

It is a communication between two processes where a sender (source process) sends message to a receiver (destination process)

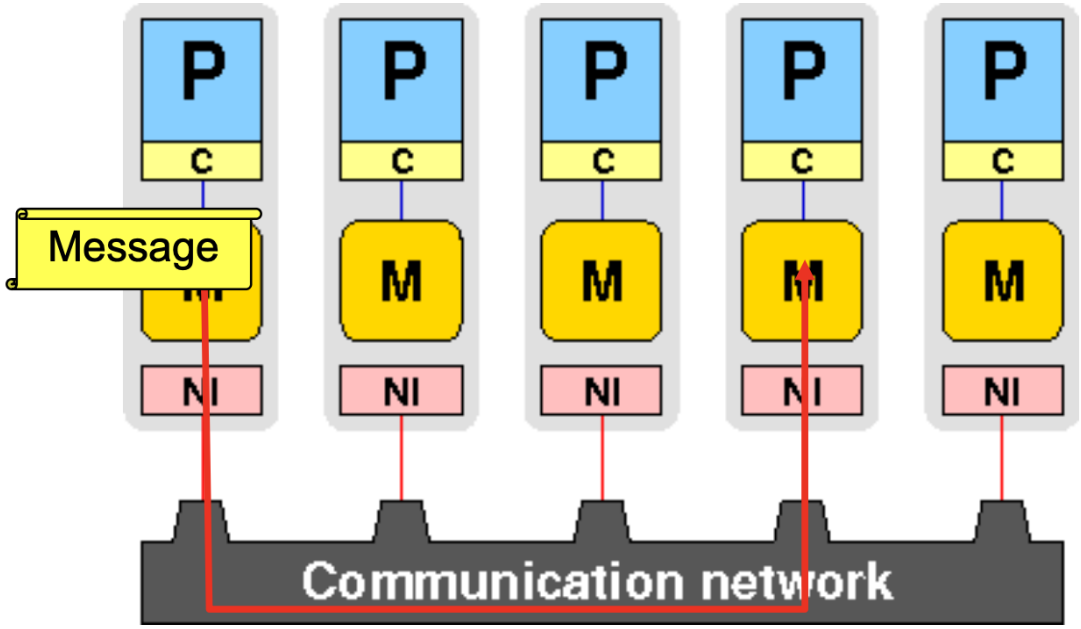
- Procedure (C/C++ binding, Fortran binding, Fortran 2008 binding)
- Message data
  - Buffer (address)
  - Datatype (basic or derived?)
  - Count (number of elements, not bytes)
- Message envelope
  - Source
  - Destination
  - Tag

Point-to-point communication: message envelope

- Which process is sending the message?
- Where is the data on the sending process?
- What kind of data is being sent?
- How much data is there?

- Which process is receiving the message?
- Where should the data be left on the receiving process?
- How much data is the receiving process prepared to accept?

- Sender and receiver must pass their information to MPI separately



MPI point-to-point communication

- Processes communicate by sending and receiving messages
- MPI message: array of elements of a particular type

sender

receiver

- **Data types**
  - **Basic**
  - **MPI derived types**

MPI_SEND

- C/C++ binding:

```
#include <mpi.h>
int MPI_Send(const void *buf, int count, MPI_Datatype datatype,
int dest,int tag, MPI_Comm comm)
```

- buf: address of the first entry of the buffer to be sent
- count: number of elements to be sent (note that it is not bytes!)
- datatype: type of the data
- dest: rank of the destination process within the communicator comm
- tag: nonnegative integer which is additional transferred with the message
- Usage: the program can categorize the messages to identify one set to another

MPI_RECV

- C/C++ binding:

```
#include <mpi.h>
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
int source,int tag, MPI_Comm comm,MPI_Status *status)
```

- buf: address of the first entry of the buffer in which the data will be stored
  - Must be large enough otherwise an overflow error occurs!
- count: The length of the received message must be less than or equal to the length of the receive buffer. The count argument indicates the maximum length of a message; the actual length of the message can be determined with MPI_Get_count(MPI_Status* status, MPI_Datatype datatype, int* count).
- source: rank of the source (sender) process within the communicator comm
- status: contains information about the received message, to be explained!

Summary of Basic MPI API Calls

Although MPI functions, there are several basic ones。

- MPI_Init let's get going
- MPI_Comm_size how many are we?
- MPI_Comm_rank who am I?
- MPI_Send send data to someone else
- MPI_Recv receive data from some-/anyone
- MPI_Get_count how many items have I received?
- MPI_Finalize finish off

- Standard send/receive calls provide most simple way of point-to-point communication
- Send/receive buffer may safely be reused after the call has completed
- MPI_Send must have a specific target/tag, MPI_Recv does not

Quiz

- Which of the following is correct?
  - a. There is a mechanism for automatic workload distribution in MPI
  - b. MPI allows for data transfer through a communication network
  - c. In MPI, workload can be split among processes according to their ranks
  - d. To execute an application, the MPI standard prescribes a startup procedure

Answer: b, c

- **Is the rank of a process within a communicator unique?**
  - a. Yes.
  - b. No

Answer: a

- **Does count in MPI_Send and MPI_Recv determine the number of bytes in the point-to-point communication??**
  - a. Yes.
  - b. No

Answer: b

Exercise: MPI “Hello World!” in C

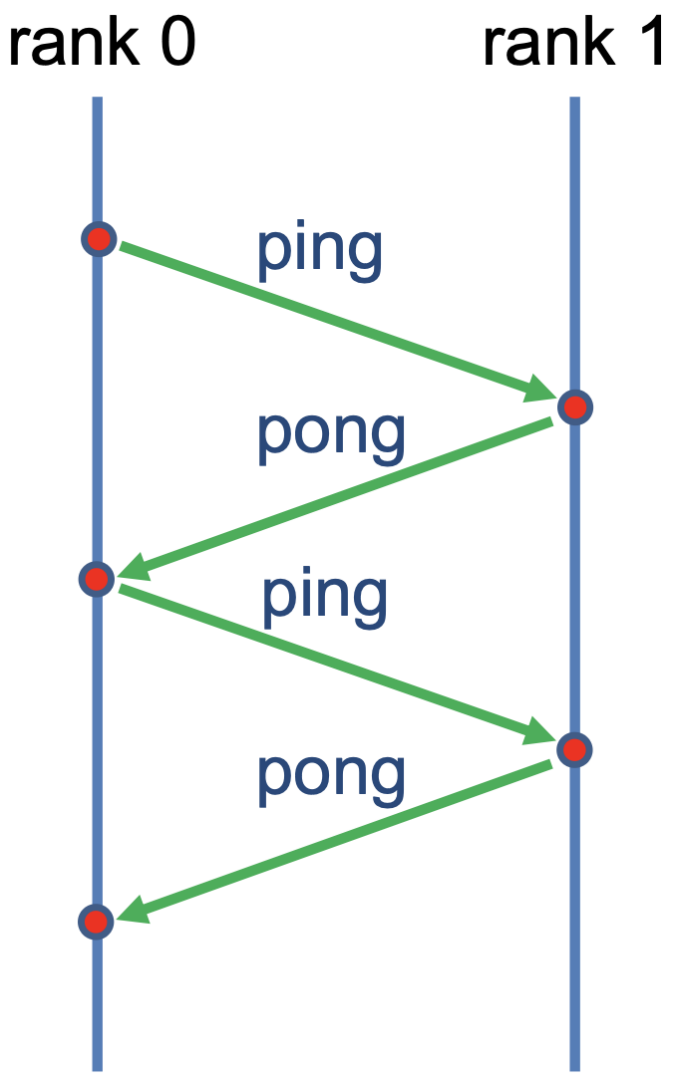
```
/*
 * hello.c - MPI "hello, world" program
 */
#include <mpi.h>
int main(char argc, char **argv) {
        int rank, size;

        MPI_FIXME(FIXME, FIXME);

        MPI_Comm_FIXME(MPI_COMM_WORLD, &size);
        MPI_Comm_rank(FIXME, FIXME);

        printf("Hello World! I am %d of %d\n", rank, size);

        MPI_FIXME();
}
```

Blocking communication

- Definition: a blocking communication does not return until the message data and envelope have been safely stored away so that the sender is free to modify the send buffer after return.
- The term blocking may be confusing. Indeed based on the definition above, one can infer:
  - The call to a send procedure does not obstruct the flow of the program at that line of the code up to the completion of the communication. Therefore, a blocking sender may return when the transmission of the message may be:
    - not yet started
    - ongoing
    - completed (less likely

The Ping-Pong example

- Consider two processes with ranks 0 and 1
- Rank 0 sends a message to rank 1
- Rank 1 receives it and sends it back to rank 0
- The operation can be repeated several times.



The Ping-Pong example

- In this example, the ping-pong is done only once.
- The code in C will be shown to you. Then, think about the following questions:
  - 1. Does rank 0 receive the initial value of rank 1, if not, how can it be done?
  - 2. If we add a loop enclosing the data transfer lines of the program and repeat the operation, would the program run without any problem?
  - 3. Does the program have a deadlock problem? If not, can the problem be introduced to the program only by rearranging the orders of send/receive?

A deadlock is a scenario in which a process is trying to exchange data to another process but there is no match, e.g. it is ready to send a data but the other process is not and will not be prepared to accept or the opposite case, i.e. the process is waiting to receive but the other is not sending and will not send a matching message.

Single-round ping-pong in C

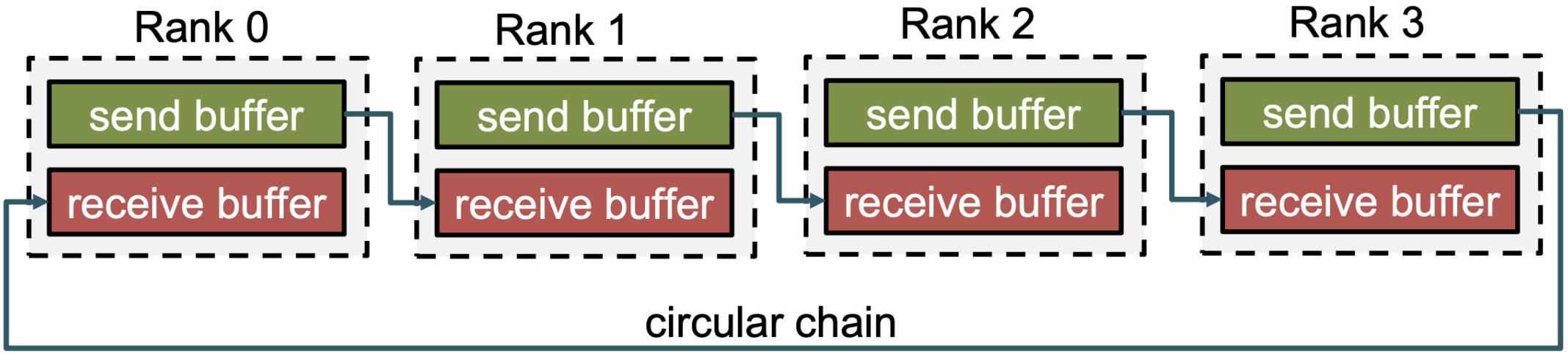
```
#include <mpi.h>
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
        int ierr, irank, nrank;
        MPI_Status status;
        double d=0.0;
        ierr=MPI_Init(&argc,&argv);
        ierr=MPI_Comm_rank(MPI_COMM_WORLD,&irank);
        ierr=MPI_Comm_size(MPI_COMM_WORLD,&nrank);
        if(irank==0) d=100.0;
        if(irank==1) d=200.0;
        printf("BEFORE: nrank,irank,d = %5d%5d%8.1f\n",nrank,irank,d);
        if(irank==0) {
                MPI_Send(&d,1,MPI_DOUBLE,1,11,MPI_COMM_WORLD);
                MPI_Recv(&d,1,MPI_DOUBLE,1,22,MPI_COMM_WORLD,&status);
        }
        else if(irank==1) {
                MPI_Recv(&d,1,MPI_DOUBLE,0,11,MPI_COMM_WORLD,&status);
                MPI_Send(&d,1,MPI_DOUBLE,0,22,MPI_COMM_WORLD);
        }
        printf("AFTER: nrank,irank,d = %5d%5d%8.1f\n",nrank,irank,d);
        ierr=MPI_Finalize();
}
```

Inspection on question 3

- Let's consider two changes in the solution code:
  - 1. Both processes call first MPI_SEND and then MPI_RECV
  - 2. The buffer is not a scalar but an array whose length is determined at run time as an argument:

```
$mpirun-n 2 ./a.out 10     # OK
$mpirun-n 2 ./a.out 100    # OK
$mpirun-n 2 ./a.out 1000   # OK
$mpirun-n 2 ./a.out 10000  # OK
$mpirun-n 2 ./a.out ???????? # at some array length DEADLOCK occurs
```

Example: Shift operation across a chain of processes



- Simplistic send/recv
  - pairing is not reliable

```
//my left neighbor
left=(rank-1)%size;
//my right neighbor
right=(rank+1)%size;
MPI_Send(sendbuf,n,type,right,tag,comm);
MPI_Recv(recvbuf,n,type,left,tag,comm,status);
```

Point-to-Point Communication MPI_SENDRECV

- C/C++ binding:

```
int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, int dest, int sendtag,
                 void *recvbuf, int recvcount, MPI_Datatype recvtype, int source, int recvtag,
                 MPI_Comm comm, MPI_Status *status)
```

- sendbuf: address of the first entry of the buffer to be sent
- sendcount: number of elements to be sent
- sendtype: type of the send data
- recvbuf, recvcount, recvtype: similarly for the receiving data
- dest: rank of the destination process within the communicator comm
- sendtag and recvtag: can have different values!

Point-to-Point Communication MPI_SENDRECV

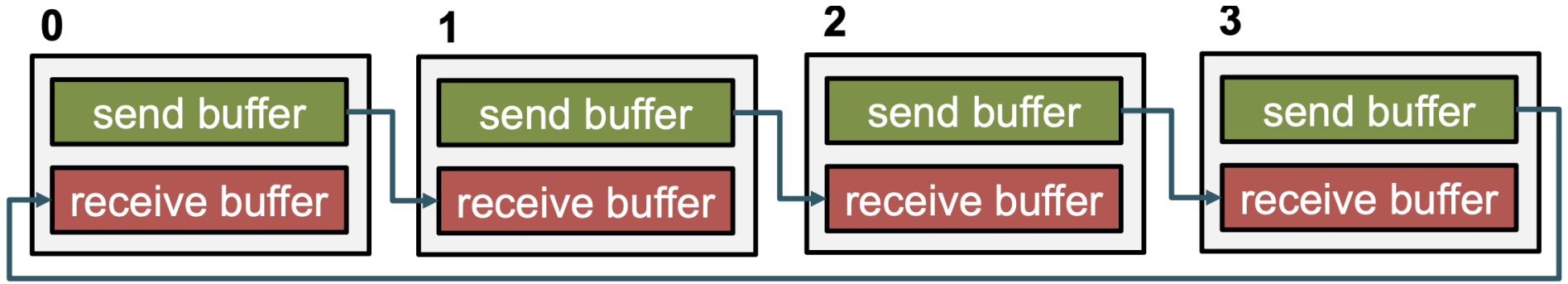
- Syntax: simple combination of send and receive arguments:

```
MPI_Sendrecv(buffer_send, sendcount, sendtype, dest, sendtag,
             buffer_recv, recvcount, recvtype, source, recvtag,
             comm, MPI_Status * status)
```

- MPI takes care, thereby no deadlocks occur:

```
// Rank left from myself
left = (rank- 1 + size) % size;
// Rank right from myself
right = (rank + 1) % size;
MPI_Sendrecv(buffer_send, n, MPI_INT, right, 0,
             buffer_recv, n, MPI_INT, left, 0,
             MPI_COMM_WORLD, status);
```

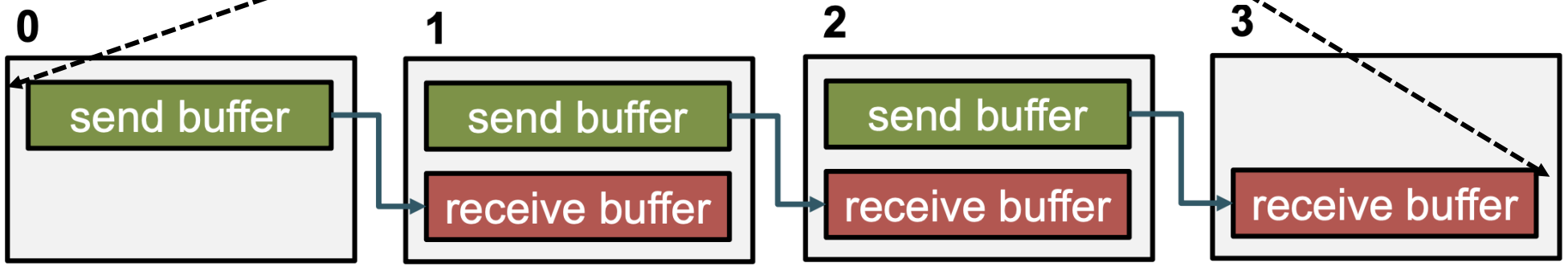
- disjoint send/receive buffers
- can have different count & data type
- blocking call



Point-to-Point Communication MPI_SENDRECV

- useful for open chains/non-circular shifts:

```
// Rank left from myself.
left = rank- 1; if (left < 0) { left = MPI_PROC_NULL; }
// Rank right from myself.
right = rank + 1; if (right >= size) {right = MPI_PROC_NULL; }
MPI_Sendrecv(buffer_send, n, MPI_INT, right, 0,
             buffer_recv, n, MPI_INT, left, 0, MPI_COMM_WORLD, &status);
```

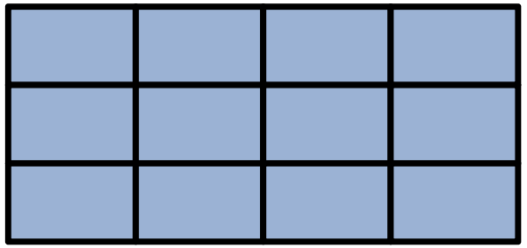


- MPI_PROC_NULL as source/destination acts as no-op
  - send/recv with MPI_PROC_NULL return immediately, buffers are not altered
- MPI_Sendrecv matches with simple *send/*recv point-to-point calls

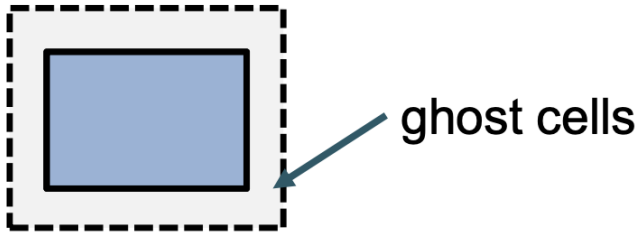
Pattern: ghost cell exchange

Many iterative algorithms require exchange of domain boundary layers

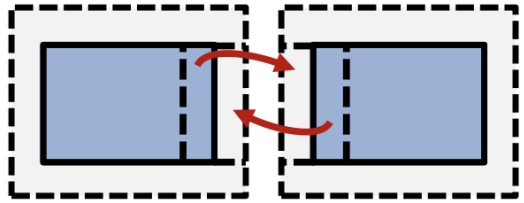
- 2D domain distributed to ranks (here 4 x 3), each rank gets one tile



- Each rank's tile is surrounded by ghost cells, representing the cells of the neighbors



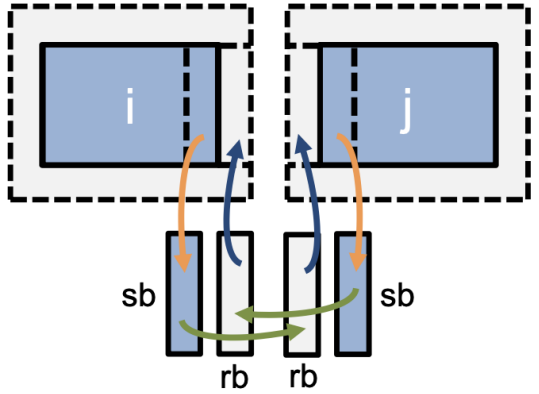
- After each sweep over a tile perform ghost cell exchange, i.e., update ghost cells with new values of neighbor cells



- Possible implementation:
  - 1. copy new data into contiguous send buffer
  - 2. send to corresponding neighbor receive new data from same neighbor
  - 3. copy new data into ghost cells

```
MPI_Sendrecv(
sb, …, j,
rb, …, j, …)
```

step 2




```
MPI_Sendrecv(
sb, …, i,
rb, …, i, …)
```

step 2

Combined send and recv

- MPI_Sendrecv combines a blocking send and receive into a single API call.
- Deadlocks are still possible if envelope does not match.
- Send and receive buffers must not overlap:
  - For specific cases: MPI_Sendrecv_replace

Quiz

- If you send two messages m_1 and m_2 from rank i to rank j using PtP bindings, is it possible that the transfer of m_2 overtakes, i.e., be received before m_1?
  - No, messages in PtP communication were in the past subject to nonovertaking rule. In recent MPI standards, it is the default behavior but it can be overridden.
- What are the risks of the standard send?
  - (i) Deadlock, (ii) high latency.
- Does the receive process have to know the rank of the send process and the tag of the message?
  - No, it is not necessary. One can use wild cards such as MPI_ANY_SOURCE and MPI_ANY_TAG.

Exercise: SendRecv (1)

Write a simple program where every processor sends data to the next one. You may use as a starting point the basic.c or basic.f90. The program should work as follows:

- Let ntasks be the number of the tasks.
- Every task with a rank less than ntasks-1 sends a message to task myid+1. For example, task 0 sends a message to task 1.
- The message content is an integer array of 100 elements.
- The message tag is the receiver's id number.
- The sender prints out the number of elements it sends and the tag number.
- All tasks with rank ≥ 1 receive messages. You should check the MPI_SOURCE and MPI_TAG fields of the status variable. Also check the number of elements that the process received using MPI_Get_count.
- Each receiver prints out the number of elements it received, the message tag, and the rank.
- Write the program using MPI_Send and MPI_Recv

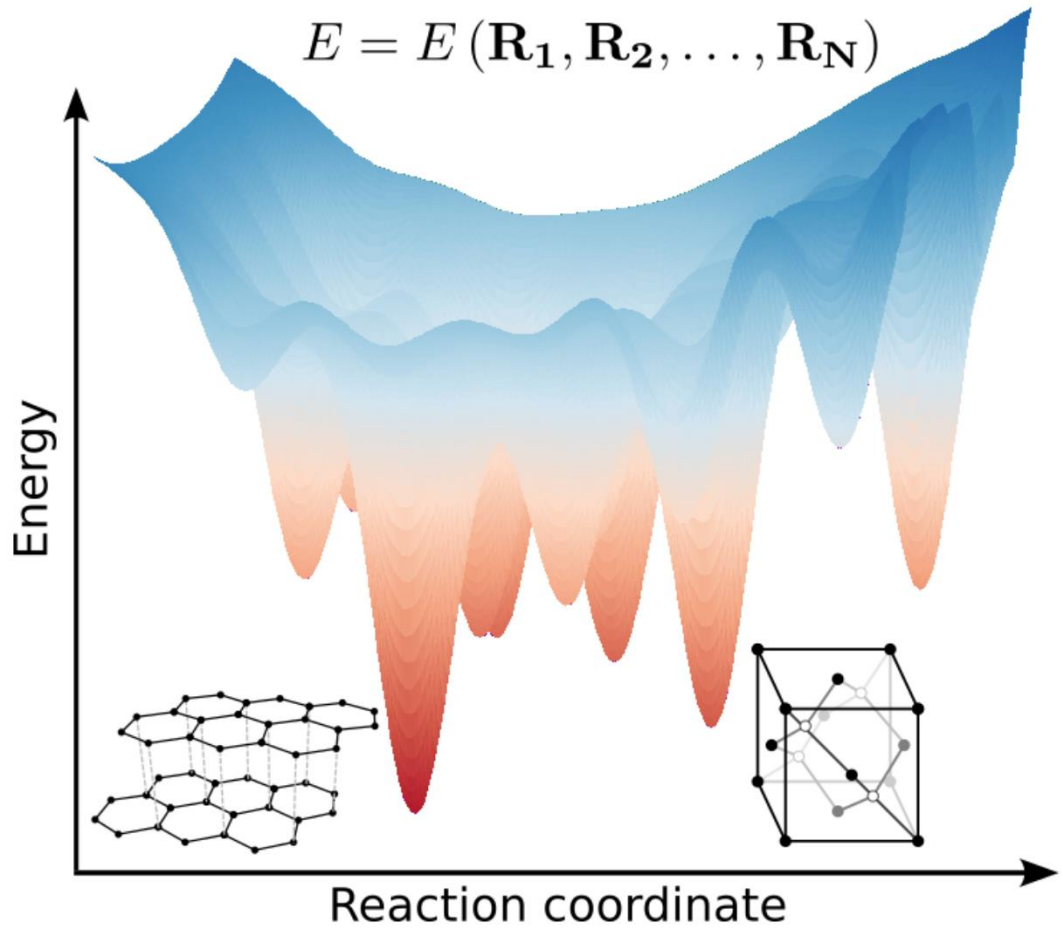
Exercise: SendRecv (2)

Find out what the program deadlock is supposed to do. Run it with two processors and see what happens:

- Why does the program get stuck ?
- Reorder the sends and receives in such a way that there is no deadlock.
- Replace the standard sends with non-blocking sends (MPI_Isend/ MPI_Irecv) to avoid deadlocking. See the man page how to use these non-blocking
- Replace the sends and receives with MPI_SENDRECV.
- In the original program set maxN to 1 and try again.

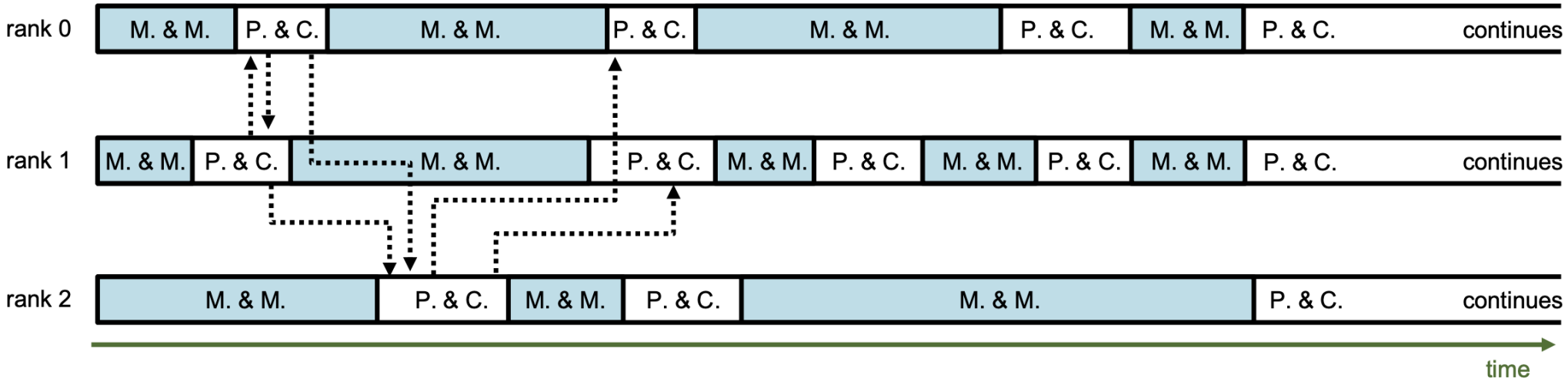
Crystalline and Molecular Structures

- Challenges in structure search: chemistry and material science
  - Local minimization
    - Computational cost: variable
  - Many local minima
    - Exponential increase with respect to system size
  - Global optimization methods
    - Deterministic and stochastic walker
- Parallel computers:
  - Each MPI process taking care of a walker



Work load imbalance

- Walker
  - Move and Minimization (M&M)
  - Processing and Communication (P&C)

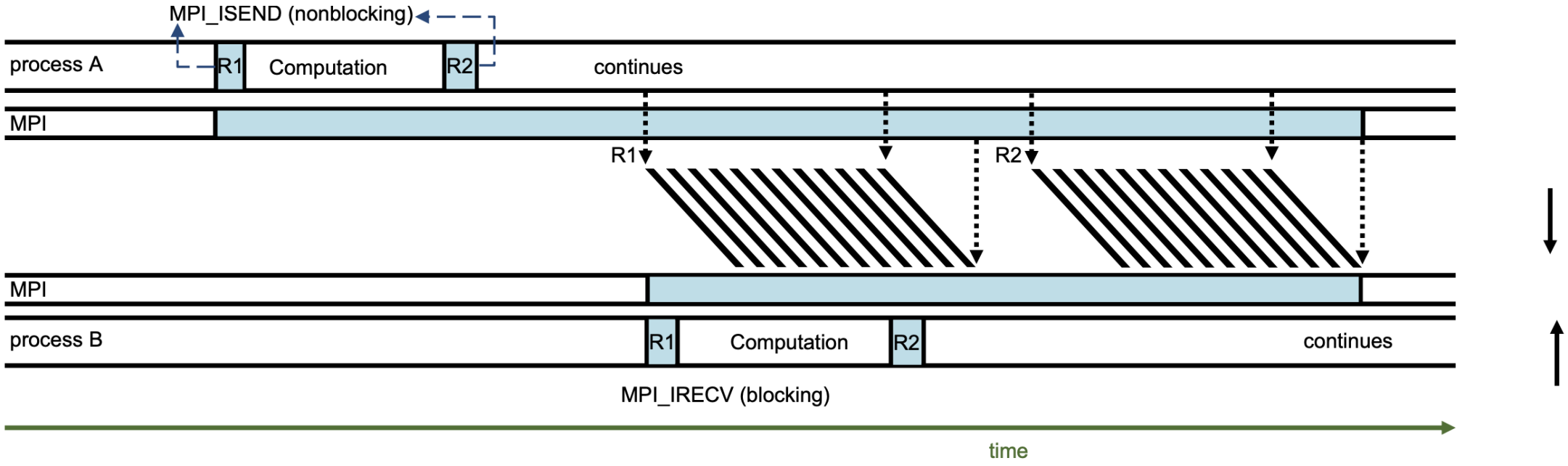


- If blocking PtP communication mode would be used:
  - will result in significant loss of resources: idle time and synchronization
  - The timelines of walkers would include long idle time of many processes

This is a prime example that blocking PtP communication is inappropriate!

Nonblocking point-to-point communication

- Call to a nonblocking send/recv procedure returns straight away. It avoids synchronization so that the following opportunities can be exploited:
  - Avoiding certain deadlocks
  - Avoid idle time: Overlapping commun. and computation
  - Truly bidirectional communication.



Standard nonblocking send/receive

```
MPI_Isend(sendbuf, count, datatype, dest, tag, comm, MPI_Request * request);
MPI_Irecv(recvbuf, count, datatype, source, tag,comm, MPI_Request * request);
```

- request: pointer to variable of type MPI_Request, will be associated with the corresponding operation
- Do not reuse sendbuf/recvbufbefore MPI_Isend/MPI_Irecv has been completed!!!
  - Return of call does not imply completion
- MPI_Irecv has no status argument
  - obtained later during completion via MPI_Wait*/MPI_Test*
- Completion
  - Return of MPI_I*call does not imply completion
  - Check for completion via MPI_Wait* / MPI_Test*
  - Semantics identical to blocking call after successful completion

Test for completion

- Blocking
  - MPI_Wait*: Wait until the communication has been completed and buffer can safely be reused
- Nonblocking
  - MPI_Test*: Return true (false) if the communication has (not) completed
- Despite the naming, the modes both pertain to nonblocking point-to-point communication!

Test for completion – single request

Test one communication handle for completion

```
MPI_Wait(MPI_Request * request, MPI_Status * status);
MPI_Test(MPI_Request * request, int * flag, MPI_Status * status);
```

- request: request handle of type MPI_Request
- status: status object of type MPI_Status (cf. MPI_Recv)
- flag: variable of type int to test for success

```
MPI_Request request;
MPI_Status status;

MPI_Isend(
send_buffer, count, MPI_CHAR,
dst, 0, MPI_COMM_WORLD, &request);

// do some work…
// do not use send_buffer

MPI_Wait(&request, &status);

// use send_buffer
```

```
MPI_Request request;
MPI_Status status;
int flag;
MPI_Isend(
send_buffer, count, MPI_CHAR,
dst, 0, MPI_COMM_WORLD, &request);

do {
  // do some work…
  // do not use send_buffer
  MPI_Test(&request, &flag,  &status);
} while (!flag)

// use send_buffer
```

Wait for completion – all requests in a list

```
MPI_Waitall(int count, MPI_Request requests[], MPI_Status statuses[]);
MPI_Testall(int count, MPI_Request requests[], int *flag, MPI_Status statuses[]);
```

- MPI can handle multiple communication requests
- Wait/Test for completion of multiple requests
- Waits for/Tests if all provided requests have been completed

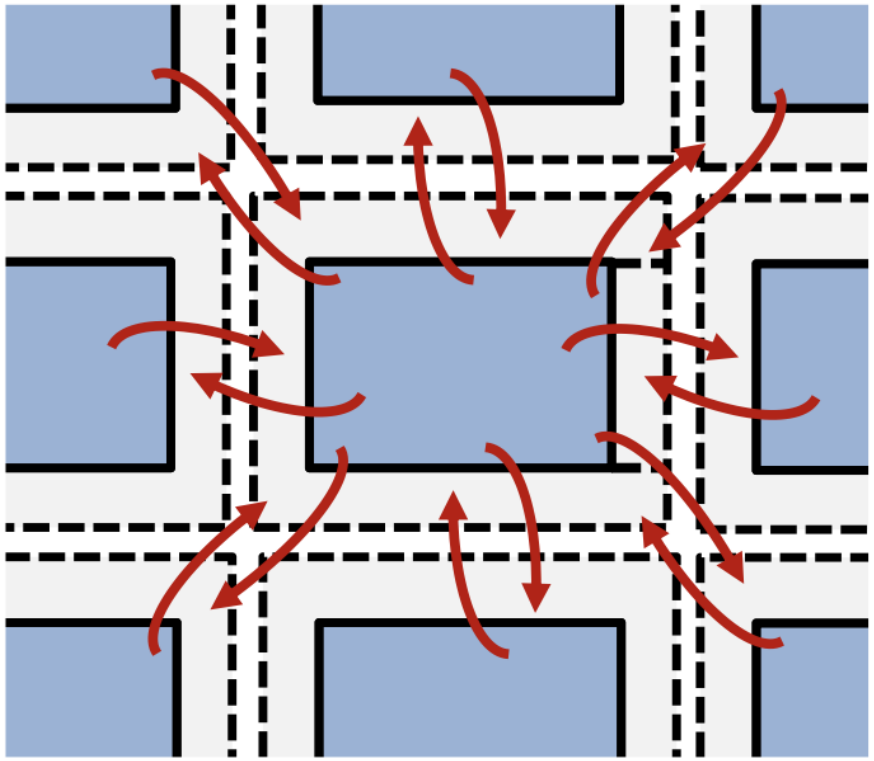
```
MPI_Request requests[2];
MPI_Status statuses[2];

MPI_Isend(send_buffer, …, &(requests[0]));
MPI_Irecv(recv_buffer, …, &(requests[1]));

// do some work…
MPI_Waitall(2, requests, statuses)
// Isend & Irecv have been completed
```

Arrays of requests and statuses

number of elements in the arrays

Ghost cell exchange with nonblocking MPI

Ghost cell exchange with nonblocking send/recv with all neighbors at once



- Possible implementation:
  - 1. Update cells that need the halo
  - 2. Copy new data into contiguous send buffers
  - 3. Start nonblocking receives/sends from/to
  - 4. corresponding neighbors
  - 5. Update local cells that do not need halo cells
  - 6. for boundary conditions ("bulk update")
  - 7. Wait with MPI_Waitall for all obtained
  - 8. requests to complete
  - 9. Copy received halo data into ghost cells

Opportunity to overlap communication (steps 3-5) with bulk update (MPI implementation permitting)

Wait for completion– one or several requests out of a list

- Wait for/Test if exactly one request among many has been completed

```
MPI_Waitany(int count, MPI_Request requests[], int * idx, MPI_Status * status);
MPI_Testany(int count, MPI_Request requests[], int * idx, int * flag, MPI_Status * status);
```

- **Wait for/Test if at least one request among many has been completed**

```
MPI_Waitsome(int incount, MPI_Request requests[], int * outcount, int indices[], MPI_Status
statuses[]);
MPI_Testsome(int incount, MPI_Request requests[], int * outcount, int indices[], MPI_Status
statuses[]);
```

```
MPI_Request requests[2];
MPI_Status status;
int finished = 0;
MPI_Isend(send_buffer, …, &(requests[0]));
MPI_Irecv(recv_buffer, …, &(requests[1]));
do {
  // do some work…
  MPI_Testany(2, requests, &idx, &flag, &status);
  if (flag) { ++finished; }
} while (finished < 2);
```

- completed requests are automatically set to MPI_REQUEST_NULL
- completed requests: requests[idx]

Nonblocking point-to-point communication

- Standard nonblocking send/recv MPI_Isend()/MPI_Irecv()
  - Return of call does not imply completion of operation
  - Use MPI_Wait*() / MPI_Test*() to check for completion using request handles
- All outstanding requests must be completed!
- Potentials
  - Overlapping of communication with work (not guaranteed by MPI standard)
  - Overlapping send and receive
  - Avoiding synchronization and reducing idle times
- Caveat: Compiler does not know about asynchronous modification of data

Quiz

- Every nonblocking send or receive requires a subsequent MPI_Wait* or MPI_Test* call?
  - Answer: Correct
- Can MPI_Isend be matched with blocking receive (MPI_Recv)?
  - Yes.
- Which one is not a certain benefit of using nonblocking MPI point-to-point calls?
  - a. Overlapping send and receive
  - b. Avoiding idle times
  - c. Overlapping of communication with work
  - Answer: c.

Timing MPI Programs

- **MPI_WTIME** returns a floating-point number of seconds, representing elapsed wall-clock time since some time in the past

```
double MPI_Wtime( void )
DOUBLE PRECISION MPI_WTIME( )
```

- **MPI_WTICK** returns the resolution of **MPI_WTIME** in seconds It returns, as a double precision value, the number of seconds between successive clock ticks.

```
double MPI_Wtick( void )
DOUBLE PRECISION MPI_WTICK( )
```

Exercise: Parallel Sum

1. Parallelize the sum.c program with MPI
   - The relevant MPI commands can be found back in the README
   - run this program with **two** MPI-tasks

2. Use **MPI_status** to get information about the message received
   - print the count of elements received

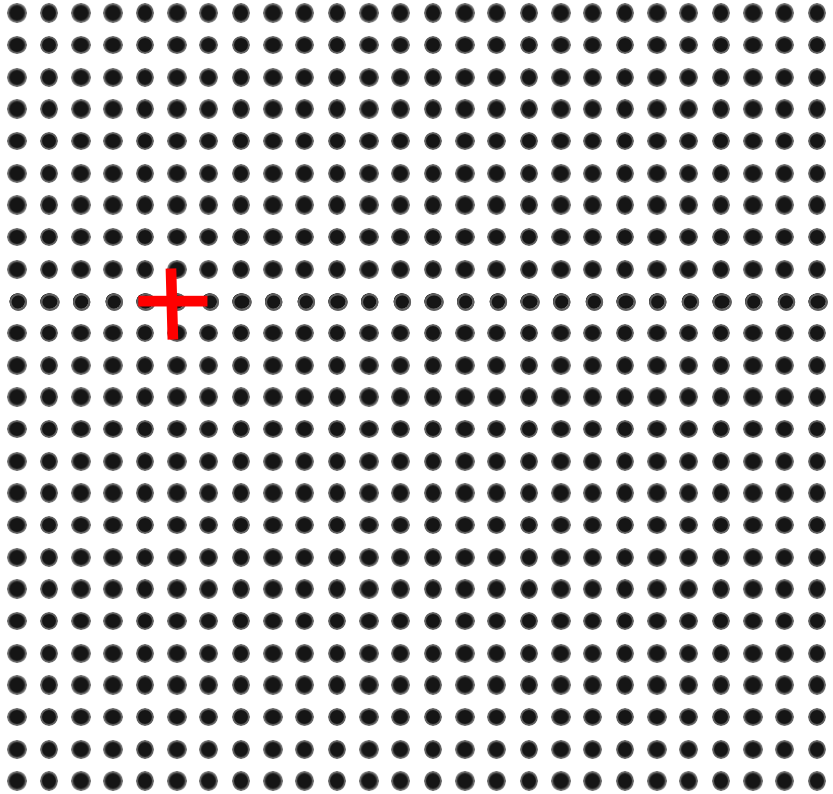
3. Using **MPI_Probe** (int source, int tag, MPI_Comm comm, int *flag, MPI_Status* status) to find out the message size to be received
   - Allocate an arrays large enough to receive the data
   - call MPI_Recv()

Exercise: Stencil

- Many scientific applications involve the solution of partial differential equations (PDEs)
- Many algorithms for approximating the solution of PDEs rely on forming a set of difference equations
  - Finite difference, finite elements, finite volume
- The exact form of the differential equations depends on the particular method
  - From the point of view of parallel programming for these algorithms, the operations are the same
- Five-point stencil is a popular approximation solution

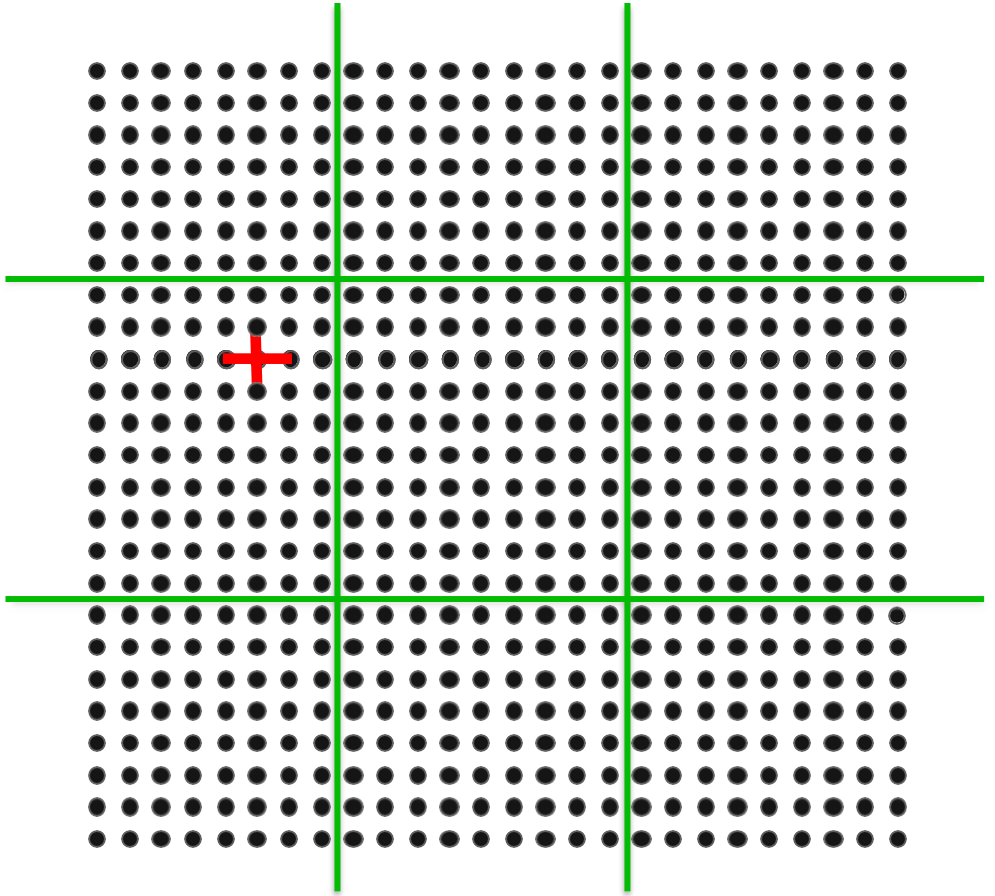
The Global Data Structure

- Each circle is a mesh point
- Difference equation evaluated at each point involves the four neighbors
- The red "plus" is called the method's stencil
- Good numerical algorithms form a matrix equation Au=f; solving this requires computing Bv, where B is a matrix derived from A. These evaluations involve computations with the neighbors on the mesh.

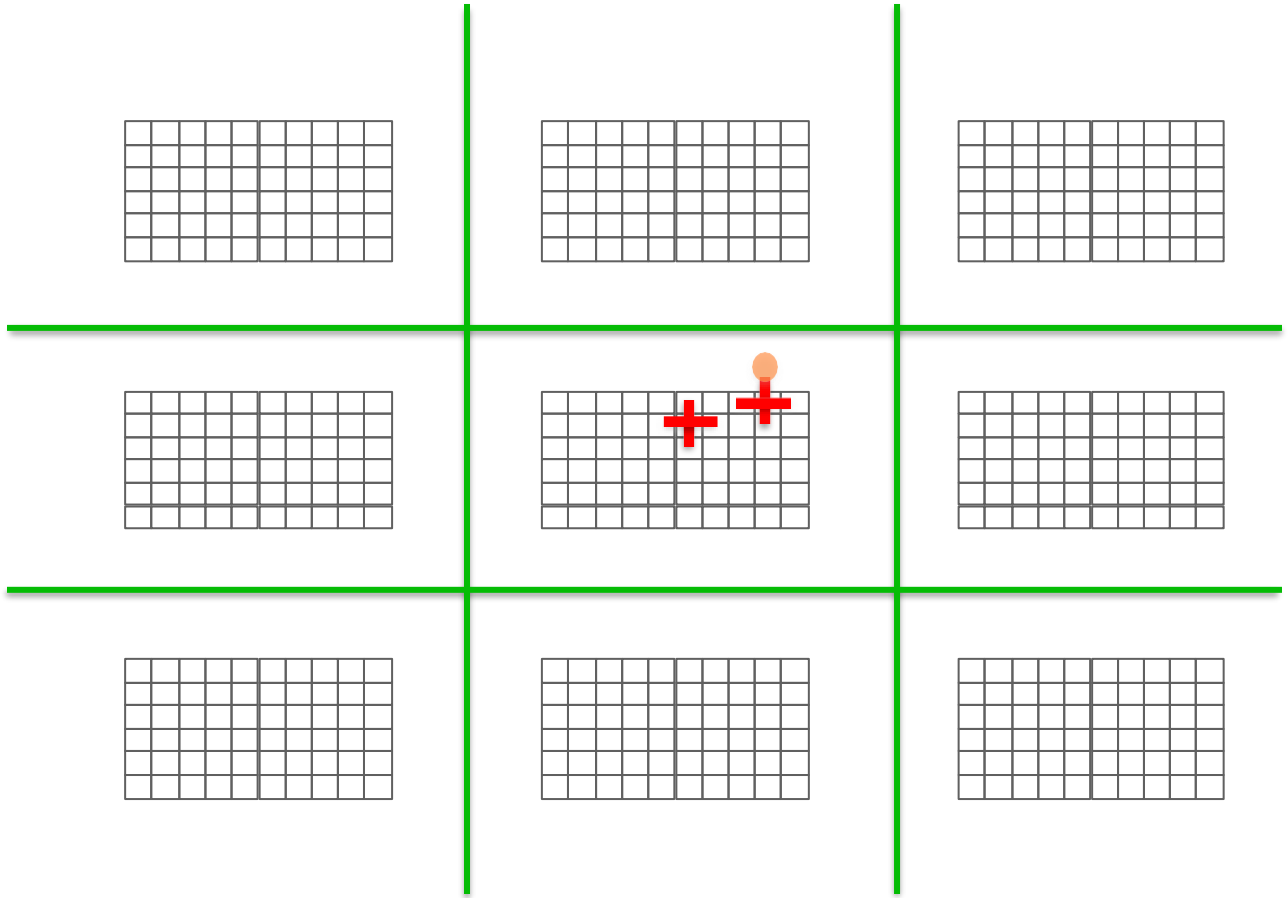


The Global Data Structure

- Each circle is a mesh point
- Difference equation evaluated at each point involves the four neighbors
- The red "plus" is called the method's stencil
- Good numerical algorithms form a matrix equation Au=f; solving this requires computing Bv, where B is a matrix derived from A. These evaluations involve computations with the neighbors on the mesh.
- Decompose mesh into equal sized (work) pieces

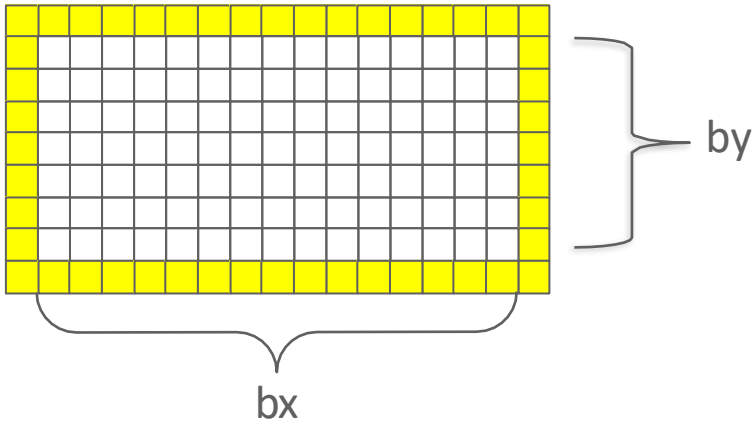


Necessary Data Transfers

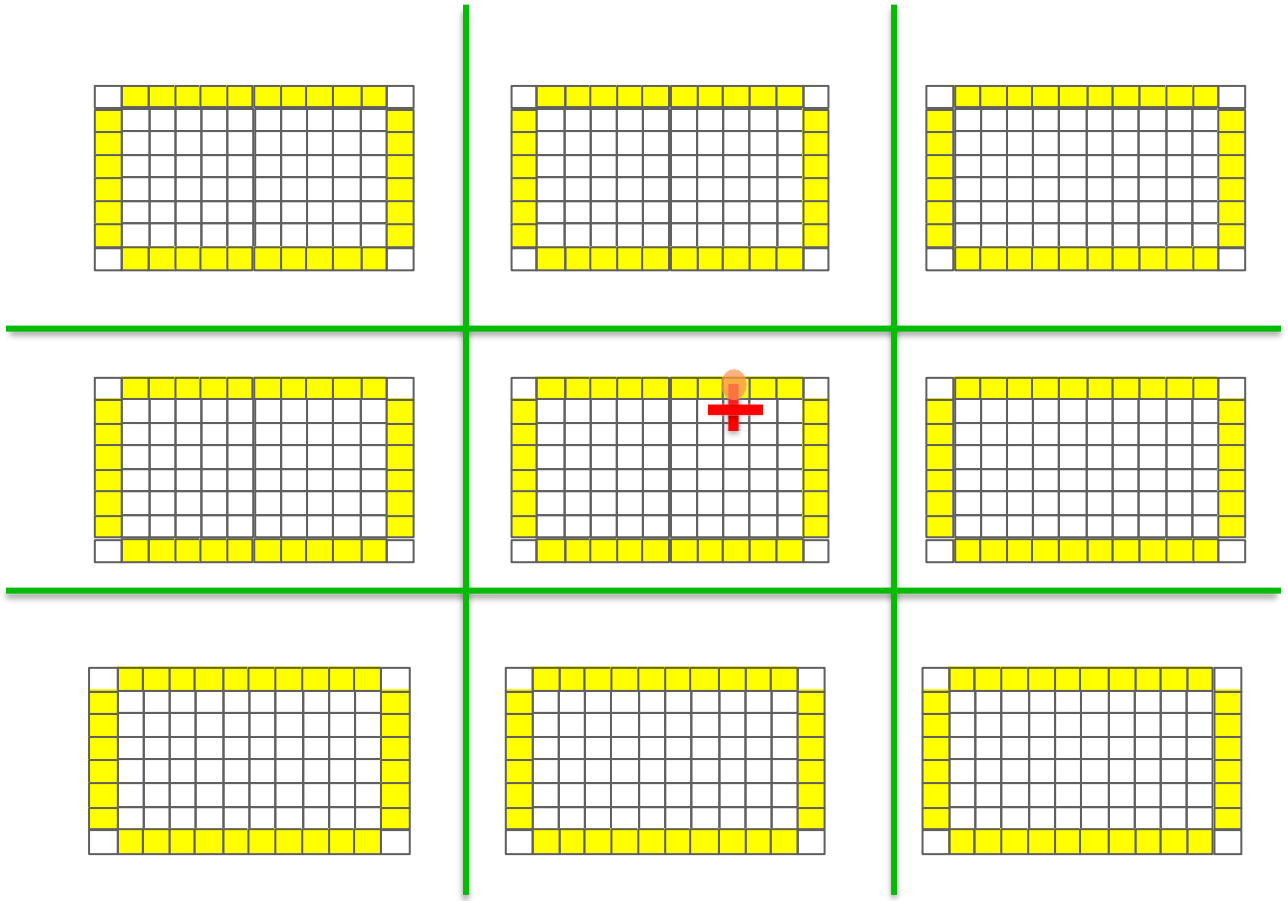


The Local Data Structure

- Each process has its local "patch" of the global array
  - "bx" and "by" are the sizes of the local array
  - Always allocate a halo around the patch
  - Array allocated of size (bx+2)x(by+2)

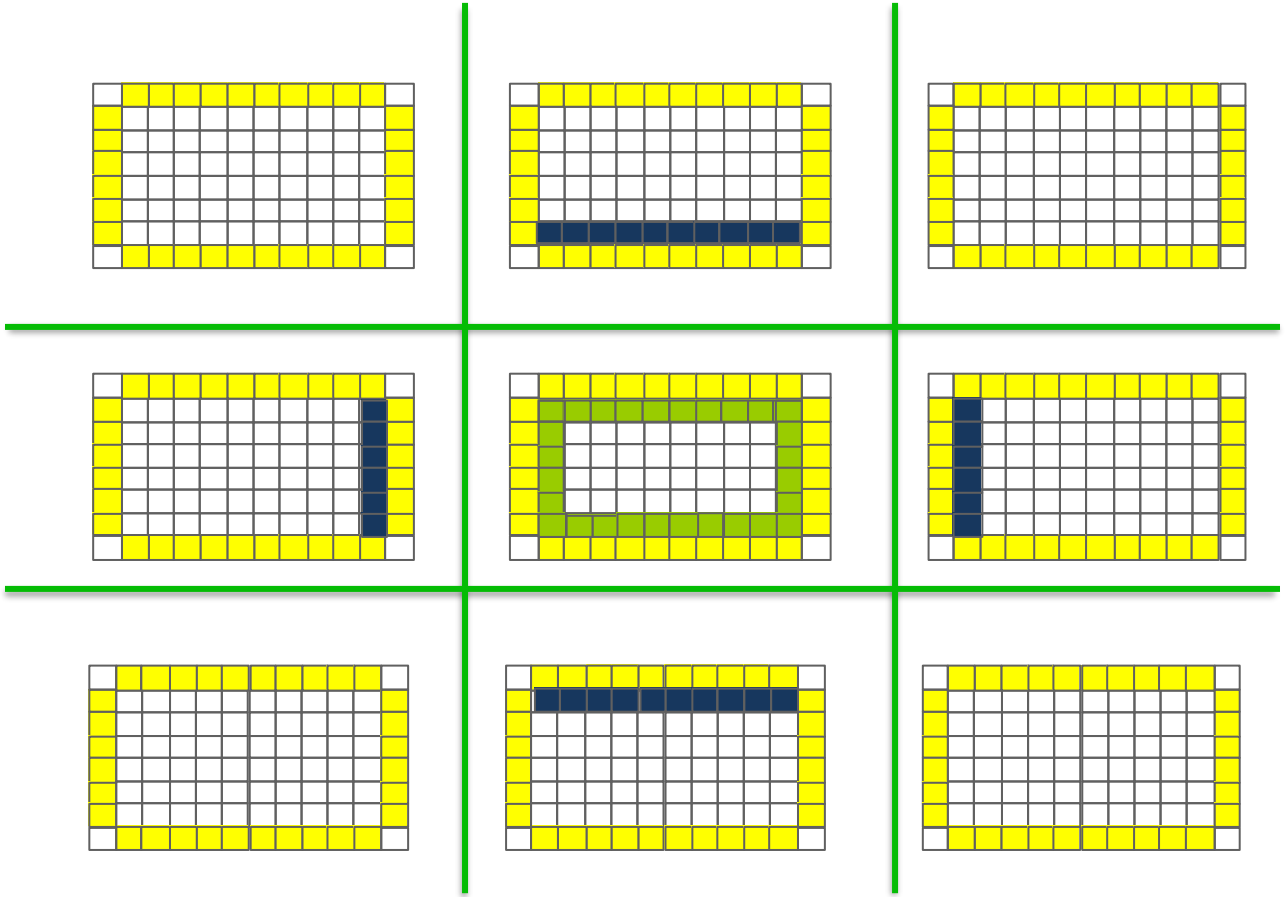


Necessary Data Transfers



Necessary Data Transfers

- Provide access to remote data through a halo exchange



(5 point stencil)

Exercise : Stencil with Nonblocking Send/recv

- nonblocking_p2p/stencil.c
- Simple stencil code using nonblocking point-to-point operations